

NeetCode 150

Every Topic Explained Like You've Never Coded Before

This guide walks through all 18 core topics behind the NeetCode 150 problem list. For each topic you'll find: a plain-English explanation, when to use it, one example question, a simple step-by-step answer with code, and a short video to watch.

Table of Contents

- 1. Arrays & Hashing
- 2. Two Pointers
- 3. Sliding Window
- 4. Stack
- 5. Binary Search
- 6. Linked List
- 7. Trees
- 8. Tries (Prefix Trees)
- 9. Heap / Priority Queue
- 10. Backtracking
- 11. Graphs
- 12. Advanced Graphs
- 13. 1-D Dynamic Programming
- 14. 2-D Dynamic Programming
- 15. Greedy
- 16. Intervals
- 17. Math & Geometry
- 18. Bit Manipulation

1. Arrays & Hashing

What is it? (in plain English)

An array is just a numbered list of items sitting next to each other in memory, like lockers in a hallway numbered 0, 1, 2, 3... A hash map (or hash set) is like a magic filing cabinet: you give it a label (a 'key'), and it instantly tells you what's filed under that label, without you having to search through every drawer.

The whole trick of 'Arrays & Hashing' problems is this: instead of re-scanning the whole list every time you want to check something (which is slow), you store what you've already seen in a hash map so you can look it up instantly (which is fast).

When do you use it?

Use this pattern whenever you need to quickly check 'have I seen this before?', count how often things appear, or find pairs/groups of items that relate to each other.

Example Question

Two Sum

You are given a list of numbers and a target number. Find the two numbers in the list that add up to the target, and return their positions (indices).

Simple Answer (explained step-by-step)

Imagine walking down the list one number at a time. For each number, ask: 'What number would I need to add to this one to reach the target?' Check your filing cabinet (hash map) to see if you've already filed that needed number. If yes — you found your pair! If no, file the current number away and keep walking.

This way you only walk through the list once, instead of comparing every number to every other number.

```
def two_sum(nums, target):
    seen = {} # number -> its index
    for i, num in enumerate(nums):
        needed = target - num
        if needed in seen:
            return [seen[needed], i]
        seen[num] = i
    return []
```

Time complexity: $O(n)$ — you only look at each number once.

Watch a Short Video

HashMaps & Dictionaries, Explained Simply

<https://www.youtube.com/watch?v=y11XNXi9dgs>

2. Two Pointers

What is it? (in plain English)

Picture two fingers pointing at two spots in a list — often one at the very start and one at the very end. You move the fingers toward each other (or in the same direction at different speeds), checking something at each step, instead of using nested loops that compare everything to everything.

This turns a slow 'compare every pair' approach into a fast single pass across the data.

When do you use it?

Use this when the list is sorted (or can be sorted) and you're looking for a pair, or when you need to compare items from both ends — like checking if a word reads the same backwards.

Example Question

Valid Palindrome

Given a string, check if it reads the same forwards and backwards (ignoring spaces, punctuation, and letter case). Example: 'A man a plan a canal Panama' is a palindrome.

Simple Answer (explained step-by-step)

Place one pointer at the very start of the string and one at the very end.

Compare the letters at both pointers (skipping over anything that isn't a letter or number). If they don't match, it's not a palindrome — stop immediately.

If they match, move the left pointer forward and the right pointer backward, and repeat until the pointers meet in the middle.

```
def is_palindrome(s):  
    s = [c.lower() for c in s if c.isalnum()]  
    left, right = 0, len(s) - 1  
    while left < right:  
        if s[left] != s[right]:  
            return False  
        left += 1  
        right -= 1  
    return True
```

Time complexity: $O(n)$ — each pointer crosses the string once.

Watch a Short Video

[Two Pointers in 7 minutes | LeetCode Pattern](#)

<https://www.youtube.com/watch?v=QzZ7nmouLTl>

3. Sliding Window

What is it? (in plain English)

Imagine a physical window frame you slide along a strip of numbers or letters. Everything currently inside the frame is your 'window'. As you slide the frame one step to the right, you drop the item that fell out the left side and pick up the new item on the right side — instead of re-checking everything inside the window from scratch every time.

This is a close cousin of Two Pointers: both pointers move in the same direction, one leading (expanding the window) and one following (shrinking it).

When do you use it?

Use this when a question asks for the 'longest', 'shortest', 'maximum', or 'minimum' something within a continuous chunk (a 'subarray' or 'substring') of the data.

Example Question

Best Time to Buy and Sell Stock

You're given a list of daily stock prices. You may buy on one day and sell on a later day. Find the maximum profit you could have made.

Simple Answer (explained step-by-step)

Keep track of the lowest price you've seen so far as you walk through the days — this is the left edge of your 'window' (the best day you could have bought).

On each day, check: if I sold today, how much profit would that be (today's price minus the lowest price so far)? Keep the biggest profit you've found.

If today's price is lower than your recorded lowest price, update the lowest price — you found a better day to 'buy'.

```
def max_profit(prices):
    lowest_price = prices[0]
    best_profit = 0
    for price in prices:
        if price < lowest_price:
            lowest_price = price
        else:
            best_profit = max(best_profit, price - lowest_price)
    return best_profit
```

Time complexity: $O(n)$ — one pass through the prices.

Watch a Short Video

[Sliding Window Technique Explained Simply | DSA Tutorial](https://www.youtube.com/watch?v=FjIPyEHZ8e8)

<https://www.youtube.com/watch?v=FjIPyEHZ8e8>

4. Stack

What is it? (in plain English)

A stack works like a stack of plates: you can only add a plate to the top, and you can only remove the plate that's currently on top. This is called LIFO — Last In, First Out. The plate you added most recently is the first one you take back off.

In code, a stack gives you two main moves: 'push' (add to the top) and 'pop' (remove from the top).

When do you use it?

Use a stack whenever something needs to be 'undone' in reverse order of how it happened — matching opening/closing brackets, undo buttons, checking nested structures, or exploring a maze path where you might need to backtrack.

Example Question

Valid Parentheses

Given a string containing only the characters () { } [], determine if the brackets are properly closed and nested. Example: '{[()]}' is valid, '{{[()]}' is not.

Simple Answer (explained step-by-step)

Walk through the string one character at a time.

Whenever you see an opening bracket ({ [, push it onto the stack — you're 'remembering' that it's waiting to be closed.

Whenever you see a closing bracket) }] , check the top of the stack: does it match the correct opening bracket? If yes, pop it off (it's been closed correctly). If no — or the stack is empty — the string is invalid.

At the very end, if the stack is completely empty, every bracket was properly closed.

```
def is_valid(s):
    pairs = {'(': ')', '[': ']', '{': '}'
    stack = []
    for char in s:
        if char in pairs:
            if not stack or stack.pop() != pairs[char]:
                return False
        else:
            stack.append(char)
    return not stack
```

Time complexity: $O(n)$ — one pass through the string.

Watch a Short Video

Stack Data Structure: Push, Pop & LIFO Explained

<https://www.youtube.com/shorts/9nAF1DmAnTI>

5. Binary Search

What is it? (in plain English)

Think about how you'd find a word in a paper dictionary. You don't start at page 1 — you flip to the middle, see if your word comes before or after that page, and throw away the half you don't need. Then you repeat on the remaining half. That's binary search.

It only works on data that's sorted — because 'throwing away half' only makes sense if you know for sure your answer isn't hiding in the half you're discarding.

When do you use it?

Use this whenever the list is sorted and you need to find something (or the boundary between two kinds of values) quickly — much faster than checking every item one by one.

Example Question

Binary Search

Given a sorted list of numbers and a target number, return the index of the target if it exists in the list, otherwise return -1.

Simple Answer (explained step-by-step)

Keep two markers: 'left' at the start of the list and 'right' at the end.

Repeatedly look at the middle item between left and right. If it's the target, you're done.

If the middle item is smaller than the target, the target must be to the right — so move 'left' just past the middle. If it's bigger, move 'right' just before the middle.

Keep shrinking the search area in half until you find the target or the area disappears.

```
def binary_search(nums, target):
    left, right = 0, len(nums) - 1
    while left <= right:
        mid = (left + right) // 2
        if nums[mid] == target:
            return mid
        elif nums[mid] < target:
            left = mid + 1
        else:
            right = mid - 1
    return -1
```

Time complexity: $O(\log n)$ — the search area is cut in half every step.

Watch a Short Video

Binary Search Algorithm in 100 Seconds

<https://www.youtube.com/watch?v=MFhxShGxHWc>

6. Linked List

What is it? (in plain English)

An array is like numbered lockers in a row — you can jump straight to locker #47. A linked list is more like a treasure hunt: each item ('node') holds its value plus a note saying where to find the next item. To reach the 47th item, you must follow the trail from the very start, one note at a time.

Because there's no 'jump straight there', linked lists are great when you're constantly adding/removing items from the middle or front, but slower when you need random access.

When do you use it?

Use this pattern for problems involving 'nodes and pointers' — reversing a sequence, detecting loops, merging sequences, or anything where items are chained together rather than indexed.

Example Question

Reverse Linked List

Given the head (starting node) of a linked list, reverse the direction of the list so the last node becomes the first.

Simple Answer (explained step-by-step)

Walk through the list one node at a time, keeping track of the node right before the current one ('previous').

At each node, before moving on, flip its arrow so it points backward to 'previous' instead of forward.

Then shuffle everything forward: 'previous' becomes the current node, and 'current' becomes the next node in line.

When you run out of nodes, 'previous' is the new head of your reversed list.

```
def reverse_list(head):
    previous = None
    current = head
    while current:
        next_node = current.next    # remember where we were going
        current.next = previous    # flip the arrow backward
        previous = current         # step forward
        current = next_node
    return previous    # new head
```

Time complexity: $O(n)$ — one pass through the list.

Watch a Short Video

Introduction to Linked List in Data Structures (very easy)

<https://www.youtube.com/watch?v=Rs1KPyb9fHY>

7. Trees

What is it? (in plain English)

A tree is like a family tree, upside down: it starts at one 'root' item at the top, which branches out into 'children'. A binary tree specifically means each item has at most two children, usually called 'left' and 'right'.

Almost every tree problem is solved with recursion: you solve the problem for a small piece (a single node), and trust that the same logic already works correctly for everything below it (its children).

When do you use it?

Use this pattern for anything hierarchical — folder structures, decision trees, organizing sorted data for fast lookup, or any 'parent has children' relationship.

Example Question

Invert Binary Tree

Given the root of a binary tree, flip it into its mirror image — every left child becomes a right child and vice versa, all the way down the tree.

Simple Answer (explained step-by-step)

If the current node doesn't exist (empty spot), there's nothing to flip — just return.

Swap the node's left and right children with each other.

Then do the exact same 'swap left and right' operation on whatever is now in the left spot, and again on whatever is now in the right spot. This is the recursion: the same tiny step repeats itself down every branch of the tree.

```
def invert_tree(root):
    if root is None:
        return None
    root.left, root.right = root.right, root.left
    invert_tree(root.left)
    invert_tree(root.right)
    return root
```

Time complexity: $O(n)$ — every node is visited exactly once.

Watch a Short Video

Binary Tree Basics: A Beginner's Guide to Understanding and Implementing Binary Trees

<https://www.youtube.com/watch?v=5p5rOZh03pg>

8. Tries (Prefix Trees)

What is it? (in plain English)

A trie is a special tree built specifically for storing words, letter by letter. Imagine a tree where each path from the root spells out a word — and words that start the same way (like 'car', 'cart', 'card') literally share the same branch for 'c-a-r' before splitting off.

This is exactly how autocomplete works on your phone: it walks down the shared branch for what you've typed so far, then shows you everything that continues from there.

When do you use it?

Use this whenever you're working with a large dictionary of words and need fast 'starts with...' lookups — autocomplete, spell-checkers, or searching for word prefixes.

Example Question

Implement a Trie (Prefix Tree)

Build a data structure that supports: inserting a word, checking if an exact word exists, and checking if any word starts with a given prefix.

Simple Answer (explained step-by-step)

Each 'node' in the trie is basically a small dictionary of 'next letter' -> 'next node', plus a flag saying 'a word ends here'.

To insert a word, walk letter by letter from the root, creating a new node for any letter that doesn't have a branch yet, then mark the final letter's node as 'end of word'.

To search for a word or a prefix, walk the same way letter by letter — if any letter is missing along the way, it doesn't exist. For an exact word, also check the 'end of word' flag at the last letter.

```
class TrieNode:
    def __init__(self):
        self.children = {}  # letter -> TrieNode
        self.is_word_end = False

class Trie:
    def __init__(self):
        self.root = TrieNode()

    def insert(self, word):
        node = self.root
        for letter in word:
            if letter not in node.children:
                node.children[letter] = TrieNode()
            node = node.children[letter]
        node.is_word_end = True
```

```
def search(self, word):
    node = self._walk(word)
    return node is not None and node.is_word_end

def starts_with(self, prefix):
    return self._walk(prefix) is not None

def _walk(self, text):
    node = self.root
    for letter in text:
        if letter not in node.children:
            return None
        node = node.children[letter]
    return node
```

Time complexity: $O(L)$ per operation, where L is the length of the word/prefix.

Watch a Short Video

[Trie Data Structure Explained | Prefix Trees for Beginners](https://www.youtube.com/watch?v=SivJ4VrVcuo)

<https://www.youtube.com/watch?v=SivJ4VrVcuo>

9. Heap / Priority Queue

What is it? (in plain English)

A heap is a data structure that always keeps the smallest (or largest) item easily reachable at the top, no matter what order you added things in. Think of it like a hospital waiting room where the most urgent patient always gets seen next, even if they arrived last.

It's not fully sorted — just organized enough that grabbing the 'best' (smallest or largest) item is always fast.

When do you use it?

Use this whenever you repeatedly need the current smallest or largest item from a changing collection — task scheduling, finding the 'top K' items, or always processing the most urgent thing next.

Example Question

Kth Largest Element in a Stream

Numbers keep arriving one at a time. After each new number arrives, report the Kth largest number seen so far.

Simple Answer (explained step-by-step)

Keep a 'min-heap' (a heap where the smallest item is always on top) that holds only the K largest numbers you've seen so far.

When a new number arrives, add it to the heap. If the heap now has more than K numbers, remove the smallest one — since it's no longer in the top K.

The smallest item currently sitting in this heap is, by definition, the Kth largest number overall.

```
import heapq

class KthLargest:
    def __init__(self, k, nums):
        self.k = k
        self.heap = nums
        heapq.heapify(self.heap)
        while len(self.heap) > k:
            heapq.heappop(self.heap)

    def add(self, val):
        heapq.heappush(self.heap, val)
        if len(self.heap) > self.k:
            heapq.heappop(self.heap)
        return self.heap[0] # smallest of the top K = Kth largest overall
```

Time complexity: $O(\log K)$ per new number added.

Watch a Short Video

Heap Data Structure & Priority Queue Basics Explained | Coding Problems

<https://www.youtube.com/watch?v=la36cnXwinY>

10. Backtracking

What is it? (in plain English)

Backtracking is 'trial and error, done systematically.' You try building a solution piece by piece. The moment you realize the path you're on can't possibly work, you undo your last step ('backtrack') and try a different option instead.

Picture exploring a maze: you walk forward, and if you hit a dead end, you walk back to the last junction and try a different turn — rather than giving up entirely.

When do you use it?

Use this for problems asking you to find 'all possible' combinations, permutations, or arrangements that satisfy some rule — like all ways to arrange items, solve a puzzle, or pick valid groups.

Example Question

Subsets

Given a list of unique numbers, return every possible subset (the 'power set') — including the empty subset and the full list itself.

Simple Answer (explained step-by-step)

Go through the numbers one at a time. For each number, you make a choice: include it in the current subset, or don't.

Explore the 'include it' path fully first (adding it, then recursively deciding for the rest of the numbers), record the subset when you reach the end of the list.

Then backtrack — remove that number again — and explore the 'don't include it' path the same way.

This tries every combination of include/don't-include, covering all possible subsets.

```
def subsets(nums):
    result = []
    current = []

    def backtrack(start):
        result.append(current.copy()) # record this subset
        for i in range(start, len(nums)):
            current.append(nums[i])   # choose
            backtrack(i + 1)          # explore
            current.pop()              # un-choose (backtrack)

    backtrack(0)
    return result
```

Time complexity: $O(2^n)$ — there are 2^n possible subsets to generate.

Watch a Short Video

Backtracking Algorithm Finally Explained Properly!

<https://www.youtube.com/watch?v=hD-AguVHfC4>

11. Graphs

What is it? (in plain English)

A graph is a set of points ('nodes') connected by lines ('edges') — think of a subway map, a friend network, or a road system. Unlike a tree, a graph can have cycles and a node can connect to many others with no strict 'parent/child' order.

The two classic ways to explore a graph are BFS (Breadth-First Search — explore layer by layer, like ripples spreading outward) and DFS (Depth-First Search — commit to one path and follow it as far as possible before backtracking).

When do you use it?

Use this for anything involving networks or connections — social networks, maps, dependency chains, or 'is there a path from A to B' style questions.

Example Question

Number of Islands

You're given a 2D grid of '1's (land) and '0's (water). An island is formed by connecting adjacent land cells (up, down, left, right). Count how many separate islands exist.

Simple Answer (explained step-by-step)

Scan every cell in the grid. Whenever you find a land cell ('1') that you haven't visited yet, you've found a brand-new island — increase your island count by one.

From that cell, explore outward in all 4 directions (this is a DFS), marking every connected land cell as 'visited' so you never count it again.

Keep scanning the rest of the grid the same way until every cell has been checked.

```
def num_islands(grid):
    rows, cols = len(grid), len(grid[0])
    visited = set()

    def dfs(r, c):
        if (r < 0 or r >= rows or c < 0 or c >= cols
            or grid[r][c] == '0' or (r, c) in visited):
            return
        visited.add((r, c))
        dfs(r + 1, c)
        dfs(r - 1, c)
        dfs(r, c + 1)
        dfs(r, c - 1)

    islands = 0
    for r in range(rows):
        for c in range(cols):
```

```
        if grid[r][c] == '1' and (r, c) not in visited:
            islands += 1
            dfs(r, c)
    return islands
```

Time complexity: $O(\text{rows} \times \text{cols})$ — every cell is visited once.

Watch a Short Video

Graph Traversals: BFS and DFS Explained for Beginners!

<https://www.youtube.com/watch?v=P9cJg-JsKgA>

12. Advanced Graphs

What is it? (in plain English)

This builds on basic graphs by adding weights (a 'cost' or 'distance' to each connection) and tackling harder questions like: what's the cheapest or shortest way to get from A to B? These problems usually need smarter algorithms than plain BFS/DFS because not all edges are equal anymore.

The most famous one is Dijkstra's algorithm: always explore from the closest unvisited point next, similar to ripples spreading out but favoring the shortest paths first.

When do you use it?

Use this for weighted networks — flight prices between cities, network routing, or any 'cheapest/fastest route' question where different connections cost different amounts.

Example Question

Network Delay Time

You have a network of nodes connected by wires, each with a travel time. A signal starts at one node. Find the minimum time for the signal to reach every other node (or -1 if it can't reach everyone).

Simple Answer (explained step-by-step)

This is a direct application of Dijkstra's algorithm. Start at the source node with a travel time of 0.

Always expand outward from whichever reachable node currently has the smallest known travel time (using a min-heap makes this efficient).

Every time you reach a neighboring node through a new path, check if this path is faster than any previously found way to reach it — if so, update its 'best known time'.

Once you can no longer improve any node's time, you have the shortest time to reach every node.

```
import heapq

def network_delay_time(times, n, source):
    graph = {i: [] for i in range(1, n + 1)}
    for u, v, w in times:
        graph[u].append((v, w))

    best_time = {}
    heap = [(0, source)]  # (time_so_far, node)

    while heap:
        time, node = heapq.heappop(heap)
        if node in best_time:
            continue
        best_time[node] = time
        for neighbor, weight in graph[node]:
            if neighbor not in best_time:
```

```
        heapq.heappush(heap, (time + weight, neighbor))

    if len(best_time) != n:
        return -1
    return max(best_time.values())
```

Time complexity: $O(E \log V)$ — using a min-heap to always expand the closest node.

Watch a Short Video

[Dijkstra's Shortest Path Algorithm Explained | With Example | Graph Theory](https://www.youtube.com/watch?v=bZkzH5x0SKU)

<https://www.youtube.com/watch?v=bZkzH5x0SKU>

13. 1-D Dynamic Programming

What is it? (in plain English)

Dynamic Programming (DP) sounds intimidating but it's really just 'remember answers to smaller versions of the problem so you never solve the same small problem twice.' 1-D DP means the answer at each step depends on a simple list of previous answers (one dimension).

Think of climbing a staircase: the number of ways to reach step 10 depends only on the number of ways to reach step 9 and step 8 — you don't need to recompute those from scratch every time.

When do you use it?

Use this whenever a problem asks you to count ways, or find a minimum/maximum result, and the answer for 'today' builds directly on the answer(s) from a few steps before.

Example Question

Climbing Stairs

You're climbing a staircase with n steps. Each move, you can climb 1 or 2 steps. How many distinct ways are there to reach the top?

Simple Answer (explained step-by-step)

To reach step n , your very last move was either a 1-step move from step $(n-1)$, or a 2-step move from step $(n-2)$. So the number of ways to reach step n is simply the ways to reach $(n-1)$ plus the ways to reach $(n-2)$.

This is the exact same idea as the Fibonacci sequence! Start from the bottom (step 1 and step 2 are trivial to figure out) and build upward, reusing the two previous answers each time instead of recalculating everything.

```
def climb_stairs(n):
    if n <= 2:
        return n
    prev2, prev1 = 1, 2 # ways to reach step 1, step 2
    for step in range(3, n + 1):
        current = prev1 + prev2
        prev2 = prev1
        prev1 = current
    return prev1
```

Time complexity: $O(n)$ — one pass building up from step 1 to step n .

Watch a Short Video

[A Beginner's Guide to Dynamic Programming](https://www.youtube.com/watch?v=oNoILrFOx2k)

<https://www.youtube.com/watch?v=oNoILrFOx2k>

14. 2-D Dynamic Programming

What is it? (in plain English)

Same idea as 1-D DP, but now the answer at each step depends on TWO changing quantities instead of one — like a position on a grid (a row AND a column), or comparing two different strings letter by letter. You build up a 2-D table of remembered answers instead of a simple list.

Once you can see the pattern of 'how does this cell's answer relate to nearby cells', 2-D DP problems become a matter of carefully filling in a table.

When do you use it?

Use this for grid-based path problems, or problems that compare two sequences (like two strings) where the answer depends on combinations of positions from both.

Example Question

Unique Paths

A robot sits at the top-left corner of an $m \times n$ grid. It can only move right or down. How many unique paths are there to reach the bottom-right corner?

Simple Answer (explained step-by-step)

The number of ways to reach any cell equals the ways to reach the cell above it, plus the ways to reach the cell to its left — since those are the only two directions you could have arrived from.

The entire top row and entire left column only have 1 way to be reached (a straight line of moves), so those become your starting values.

Fill in the rest of the grid row by row, using the 'above + left' rule, and the bottom-right cell holds your final answer.

```
def unique_paths(m, n):
    # table[row][col] = number of ways to reach that cell
    table = [[1] * n for _ in range(m)]
    for row in range(1, m):
        for col in range(1, n):
            table[row][col] = table[row - 1][col] + table[row][col - 1]
    return table[m - 1][n - 1]
```

Time complexity: $O(m \times n)$ — you fill in every cell of the grid once.

Watch a Short Video

[A Beginner's Guide to Dynamic Programming](#)

<https://www.youtube.com/watch?v=oNoILrFOx2k>

15. Greedy

What is it? (in plain English)

A greedy algorithm makes the choice that looks best right now, at each individual step, and never looks back or reconsiders — hoping (and, for the right problems, proving) that this leads to the best overall result. It's the opposite of DP's 'consider all possibilities' approach.

Greedy is fast and simple, but it only works correctly on problems where you can prove that grabbing the locally-best option never hurts your final answer — it doesn't work for every problem.

When do you use it?

Use this when you need to prove (or it's given) that always taking the best immediate option leads to the best overall outcome — scheduling, coin change with 'nice' coin values, or simple optimization problems.

Example Question

Maximum Subarray (Kadane's Algorithm)

Given a list of numbers (which can include negatives), find the contiguous run of numbers (a 'subarray') that has the largest sum.

Simple Answer (explained step-by-step)

Walk through the numbers, keeping a running sum of 'the best subarray sum ending exactly at this position'.

At each number, greedily decide: is it better to add this number onto my existing running subarray, or is my running sum actually dragging me down (negative) — in which case I should just restart fresh from this number?

Keep track of the best running sum you've seen at any point — that's your answer.

```
def max_subarray(nums):
    best_sum = nums[0]
    current_sum = nums[0]
    for num in nums[1:]:
        # either extend the previous run, or start fresh here
        current_sum = max(num, current_sum + num)
        best_sum = max(best_sum, current_sum)
    return best_sum
```

Time complexity: $O(n)$ — one pass through the numbers.

Watch a Short Video

Greedy Algorithms Explained

<https://www.youtube.com/watch?v=IfQvPHGtu6Q>

16. Intervals

What is it? (in plain English)

An interval is just a 'from...to...' range, like a meeting from 2pm to 3pm. Interval problems ask you to compare, merge, or arrange a bunch of these ranges — like figuring out which meetings overlap on a calendar.

The go-to trick: sort the intervals by their start time first. Once sorted, any intervals that overlap will always end up sitting right next to each other, which makes them easy to spot in a single pass.

When do you use it?

Use this for scheduling-style problems — merging overlapping time ranges, detecting meeting conflicts, or inserting a new range into an existing sorted set of ranges.

Example Question

Merge Intervals

Given a list of intervals (each with a start and end), merge every pair that overlaps and return the resulting list of non-overlapping intervals.

Simple Answer (explained step-by-step)

First, sort all the intervals by their starting point.

Walk through them one by one, keeping a 'current merged interval' you're building.

If the next interval's start is before (or exactly at) the end of your current merged interval, they overlap — stretch the current interval's end to cover both.

If the next interval starts after your current one ends, there's no overlap — save the current merged interval as finished, and start a new one with this next interval.

```
def merge(intervals):
    intervals.sort(key=lambda pair: pair[0])
    merged = []
    for interval in intervals:
        if not merged or merged[-1][1] < interval[0]:
            merged.append(interval) # no overlap, start new
        else:
            merged[-1][1] = max(merged[-1][1], interval[1]) # stretch
    return merged
```

Time complexity: $O(n \log n)$ — dominated by the initial sort.

Watch a Short Video

[Merge Intervals - LeetCode 56 - Visually Explained](#)

<https://www.youtube.com/shorts/yrGJLhg5fNo>

17. Math & Geometry

What is it? (in plain English)

These problems don't follow one single pattern — instead they test whether you can translate a real-world spatial or numeric idea (rotating a grid, walking in a spiral, checking prime numbers) directly into precise, careful code, often working with coordinates or number properties.

The main skill here is being methodical: carefully tracking positions (like row/column) or working out the math relationship on paper before coding it.

When do you use it?

Use this whenever a problem is about grid geometry (rotating, spiraling, transposing) or number theory (primes, GCD, digit manipulation) rather than a 'named' algorithm pattern.

Example Question

Rotate Image

You're given an $n \times n$ grid representing an image. Rotate the image 90 degrees clockwise, in place (without using a separate grid).

Simple Answer (explained step-by-step)

This can be done in two simple, well-known steps. First, 'transpose' the grid — flip it along its main diagonal, so rows become columns.

Second, reverse each individual row of the transposed grid.

Doing 'transpose, then reverse each row' is mathematically equivalent to rotating the whole grid 90 degrees clockwise.

```
def rotate(matrix):
    n = len(matrix)
    # Step 1: transpose (swap matrix[i][j] with matrix[j][i])
    for i in range(n):
        for j in range(i + 1, n):
            matrix[i][j], matrix[j][i] = matrix[j][i], matrix[i][j]
    # Step 2: reverse each row
    for row in matrix:
        row.reverse()
```

Time complexity: $O(n^2)$ — every cell in the grid is touched a constant number of times.

Watch a Short Video

[Rotate a Matrix by 90 Degrees | LeetCode 48 | Easy 2-Step Algorithm](https://leetcode.com/problems/rotate-image/)

https://www.youtube.com/watch?v=0K_JxpGOOvQ

18. Bit Manipulation

What is it? (in plain English)

Every number in a computer is secretly stored as a string of 1s and 0s ('bits'). Bit manipulation means working directly with those individual 1s and 0s using special operators: AND (&), OR (|), XOR (^), NOT (~), and shifts (<<, >>).

The one 'magic trick' worth remembering: XOR-ing a number with itself always gives 0, and XOR-ing with 0 leaves a number unchanged. This makes XOR extremely useful for spotting a number that appears an odd number of times among duplicates.

When do you use it?

Use this for problems explicitly about binary representations, or ones hinting at 'can you do this in O(1) extra space' by exploiting how numbers are stored at the bit level.

Example Question

Single Number

Given a list of numbers where every number appears exactly twice, except for one number that appears only once, find that single number.

Simple Answer (explained step-by-step)

Remember the two XOR facts: a number XOR itself is 0, and a number XOR 0 is itself.

XOR every number in the list together, one after another, into a running result.

Every number that appears twice will cancel itself out completely (becoming 0 and vanishing from the result). Whatever value is left over at the end must be the number that appeared only once.

```
def single_number(nums):  
    result = 0  
    for num in nums:  
        result ^= num # duplicates cancel each other out  
    return result
```

Time complexity: O(n) — one pass through the list, O(1) extra space.

Watch a Short Video

Bit Manipulation Basics: AND, OR, XOR, NOT & Shift Operators Explained

<https://www.youtube.com/watch?v=Ny72q67Y6mQ>